

Programación III

Desarrollo backend con Spring Boot

UNIDAD 2: JDBC y Patrones de diseño

Clase N° 4

Introducción a JDBC, Patrón Singleton y MVC



Profesor:

Daniel Diaz

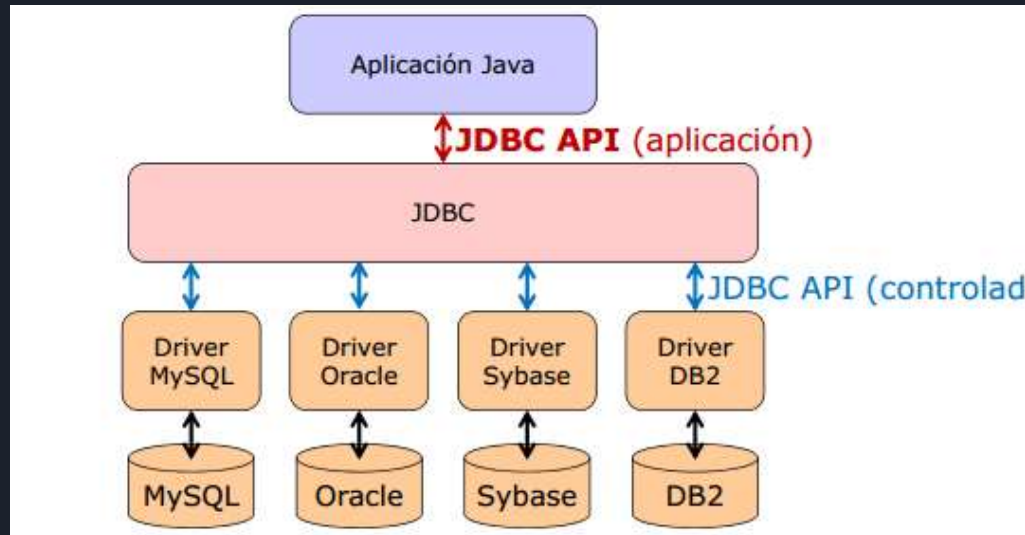


TEMAS A DESARROLLAR

- ¿Qué es JDBC?
- Arquitectura de JDBC (Driver, Connection, Statement, ResultSet)
- Descarga e instalación del conector JDBC para MySQL
- ¿Qué es un patrón de diseño?
- Patrón Singleton.
- ¿Por qué usar Singleton en la conexión a la base de datos?
- Implementación de un Singleton JDBC para la conexión.
- Diseño modular con DAO (Data Access Object)
- ¿Qué es el patrón MVC?
- Operaciones CRUD en JDBC.

¿Qué es JDBC?

JDBC (Java Database Connectivity) es una API de Java que permite conectar aplicaciones Java con bases de datos. Proporciona métodos para ejecutar consultas SQL, recuperar datos y administrar conexiones de forma eficiente.



Java Database Connectivity



¿QUÉ SON LOS PATRONES DE DISEÑO?

Son técnicas para resolver problemas recurrentes de diseño de software.



● CREACIONALES

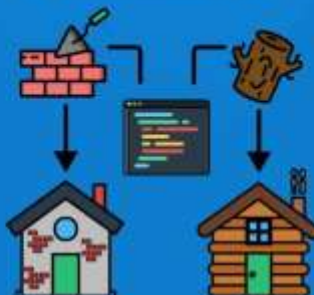
Soluciona la creación de objetos, hace un sistema independiente de cómo sus objetos son creados.



Ej. Singleton, Factory.

● ESTRUCTURALES

Describe cómo los objetos se componen para formar estructuras complejas.



Ej. Adapter, Decorator.

● COMPORTAMENTALES

Establece soluciones relacionados con el comportamiento de la aplicación respecto a la interacción entre objetos y clases.



Ej. Observer, State.

Si tienes un problema de diseño es muy probable que ya exista un patrón que lo solucione, no reinventes la rueda.



Alejandro Rodríguez
Backend Developer en EDteam

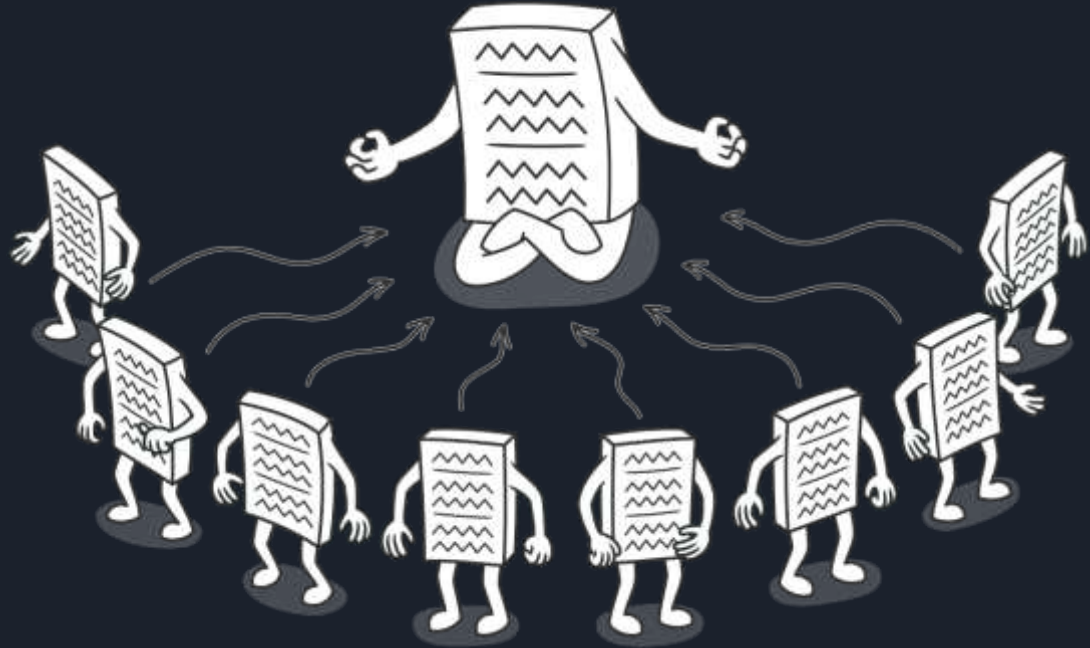
Domina los patrones de diseño de software en:
ed.team/programacion



Patrón Singleton

Propósito

Singleton es un patrón de diseño creacional que nos permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.



Problema

El patrón Singleton resuelve dos problemas al mismo tiempo, vulnerando el Principio de responsabilidad única:

1. Garantizar que una clase tenga una única instancia. ¿Por qué querría alguien controlar cuántas instancias tiene una clase? El motivo más habitual es controlar el acceso a algún recurso compartido, por ejemplo, una base de datos o un archivo.
2. Proporcionar un punto de acceso global a dicha instancia. ¿Recuerdas esas variables globales que utilizaste (bueno, sí, fui yo) para almacenar objetos esenciales? Aunque son muy útiles, también son poco seguras, ya que cualquier código podría sobrescribir el contenido de esas variables y descomponer la aplicación.

Este problema tiene otra cara: no queremos que el código que resuelve el primer problema se encuentra **disperso** por todo el programa. Es mucho más conveniente tenerlo dentro de una **clase**, sobre todo si el resto del código ya depende de ella.

Solución

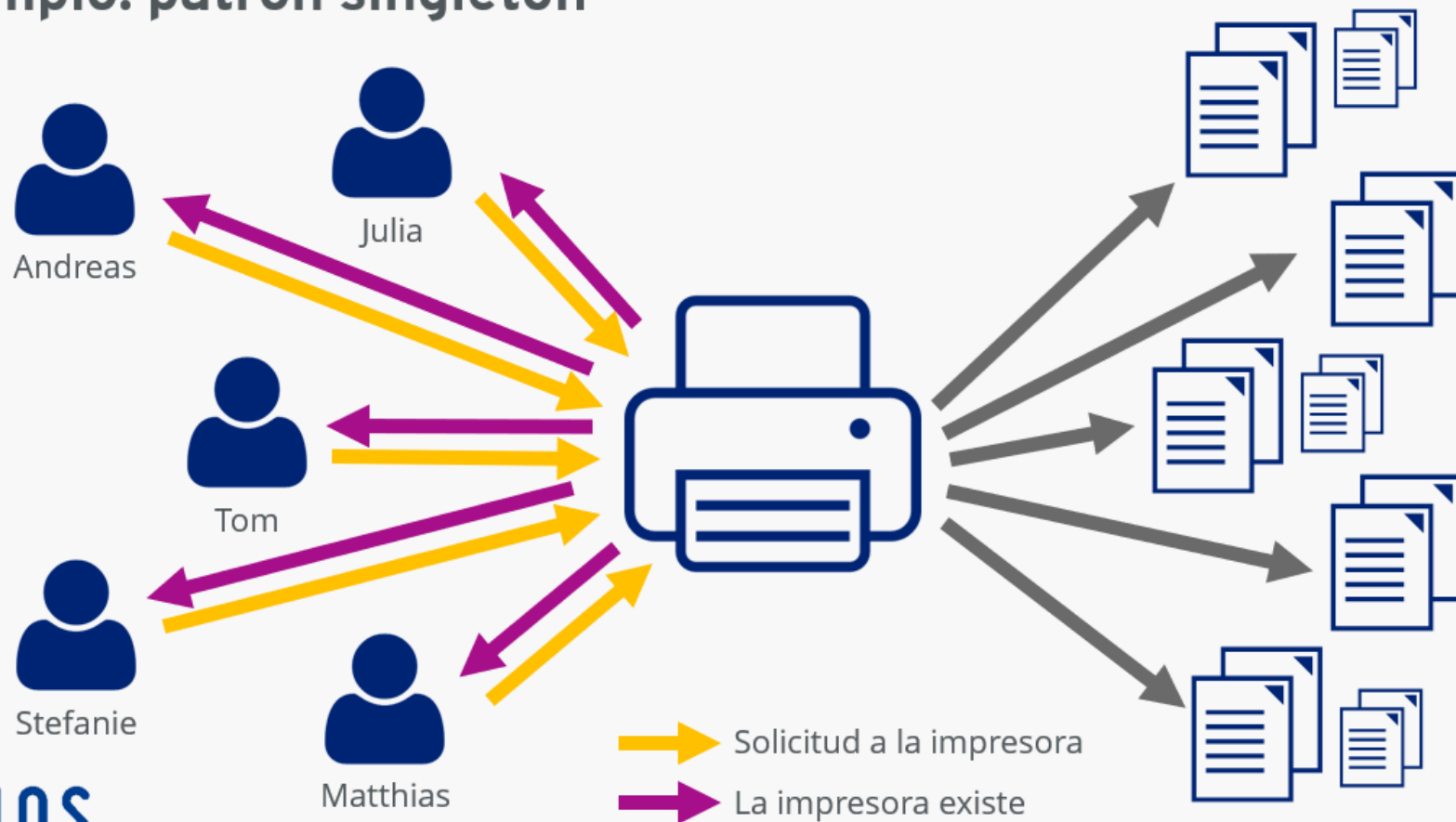
Todas las implementaciones del patrón Singleton tienen estos dos pasos en común:

1. Hacer privado el constructor por defecto para evitar que otros objetos utilicen el operador new con la clase Singleton.
2. Crear un método de creación estático que actúe como constructor.

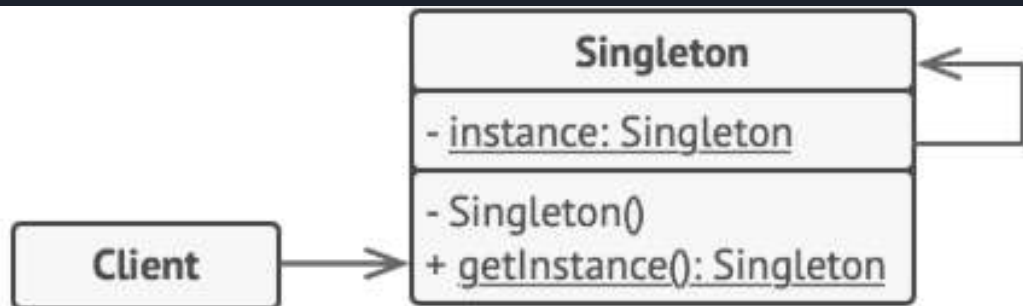
Analogía en el mundo real

El gobierno es un ejemplo excelente del patrón Singleton. Un país sólo puede tener un gobierno oficial. Independientemente de las identidades personales de los individuos que forman el gobierno, el título “Gobierno de X” es un punto de acceso global que identifica al grupo de personas a cargo.

Ejemplo: patrón singleton



Estructura



1 La clase **Singleton** declara el método estático `obtenerInstancia` que devuelve la misma instancia de su propia clase.

El constructor del Singleton debe ocultarse del código cliente. La llamada al método `obtenerInstancia` debe ser la única manera de obtener el objeto de Singleton.

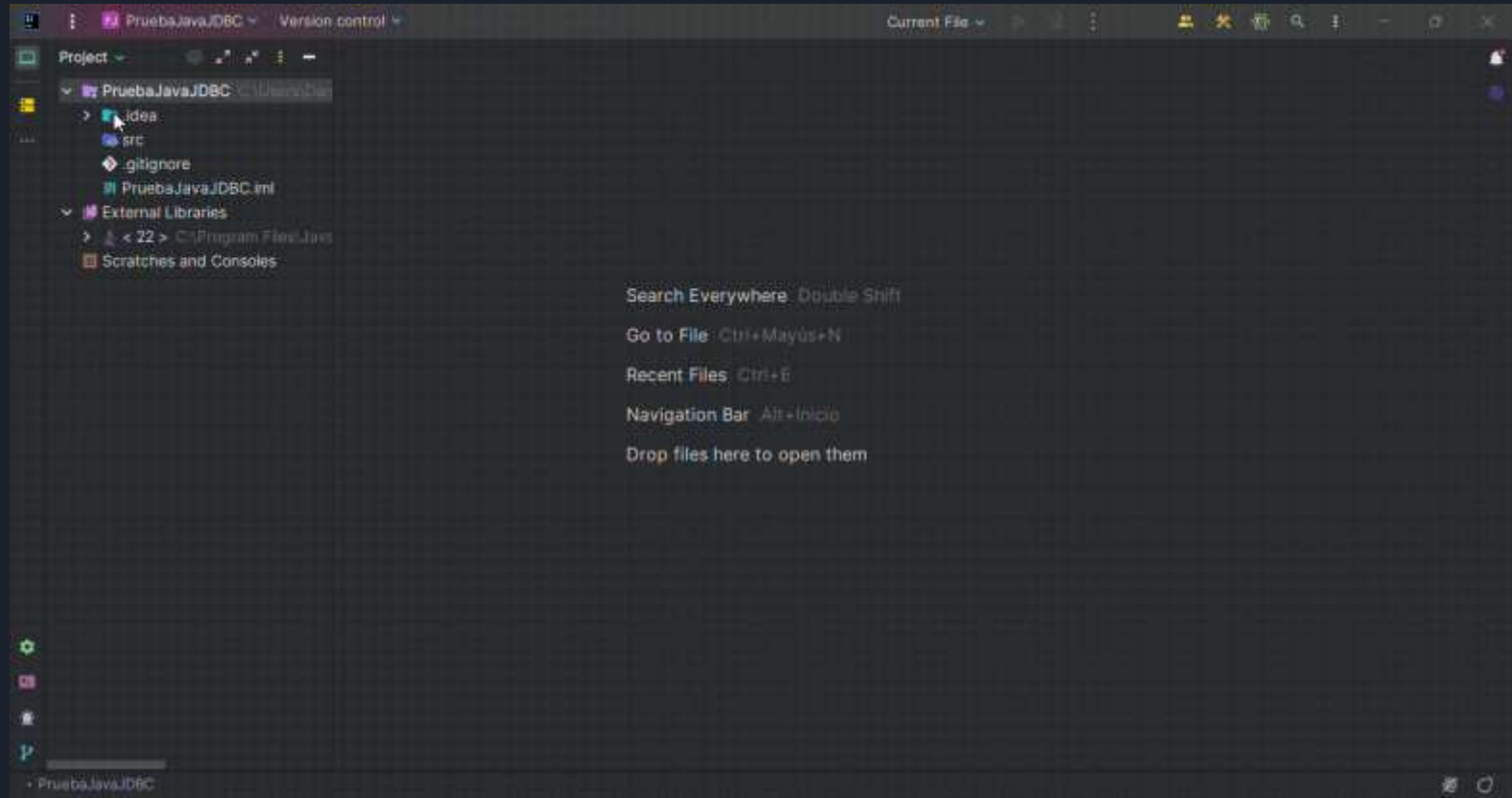
```
if (instance == null) {  
    // Nota: si estás creando una aplicación  
    // que soporte el multihilo, debes  
    // colocar un bloqueo de hilo aquí.  
    instance = new Singleton()  
}  
return instance
```

¿Cuándo aplicar el patrón Singleton?

Utiliza el patrón Singleton cuando una clase de tu programa tan solo deba tener una instancia disponible para todos los clientes; por ejemplo, un único objeto de base de datos compartido por distintas partes del programa.

JDBC + SINGLETON

Paso 1: Agregar la dependencia de JDBC



JDBC + SINGLETON

Paso 2: Implementar la conexión a la base de datos usando el patrón Singleton

Clase de conexión a una Base de Datos

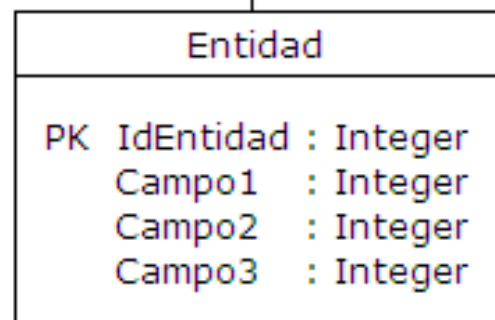
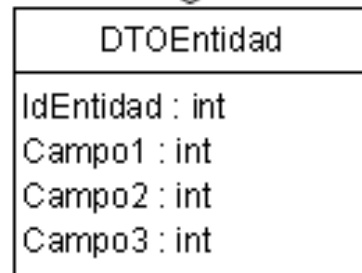
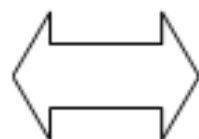
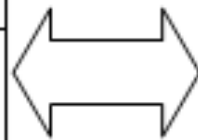
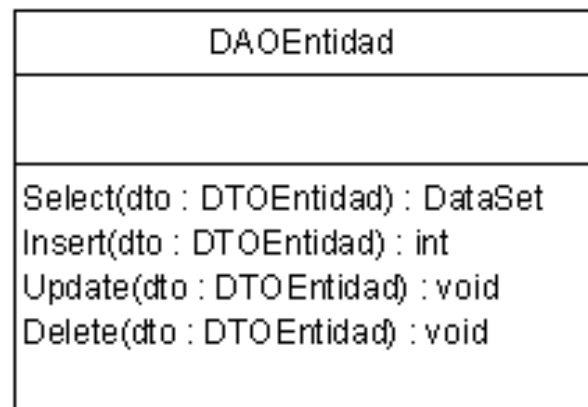
Instrucciones para testear la conexión

Todas las llamadas a las bases de datos deben estar en una capa de Acceso a Datos correspondiente a la Entidad o Tabla sobre la que se quiera recuperar información...

Patrón DAO (Data Access Object)

Es un patrón de diseño que se utiliza en el desarrollo de software para separar la lógica de acceso a datos de la lógica de negocio de una aplicación. Su propósito es proporcionar una capa de abstracción entre la aplicación y la base de datos, facilitando el **mantenimiento**, la **reutilización** del código y la **portabilidad** del sistema.

LÓGICA DE NEGOCIO



Modelo (Model)

- Representa los datos y la lógica de negocio de la aplicación.
- Se encarga de gestionar la información y las reglas de negocio.
- Puede interactuar con una base de datos u otras fuentes de datos.

Vista (View)

- Se encarga de la presentación de los datos al usuario.
- No contiene lógica de negocio, solo muestra información recibida del Modelo.

Controlador (Controller)

- Actúa como intermediario entre la Vista y el Modelo.
- Recibe las entradas del usuario desde la vista y actualiza el modelo o la vista según corresponda.

